CAPSTONE — CHECKPOINT 1 (BUILD)

PM/Product Learning Digest Agent

Backend AI Engineering Capstone · Sibgha Mursaleen

AGENT OVERVIEW

Checks a set of trusted project management and product management sources for new articles, classifies each one by skill area (e.g. Prioritization, Agile, Stakeholder Management, Roadmapping), and saves the sorted links — title, source, and original URL — into a running reading list. No summarizing or rewriting of source content; the agent's only judgment call is the skill-area classification, and every entry links back to the original article.

User: Solo use (me).

Frequency: Run on demand or on a schedule, a few times a week.

DEVIATION FROM FL-06 SPEC

> **Original spec:** a Job Application Tracker & Follow-Up Agent (Gmail + Google Sheets, OAuth-based).
>
> **Change:** pivoted to a PM/Product Learning Digest Agent.
>
> **Reason:** self-hosted n8n made Google OAuth setup a heavy, low-value time cost relative to a 10-hour build. This job needed only RSS feeds and a local file — no OAuth at all — while staying equally useful and better matched to a current, active goal (building PM/product skills).

TOOLS & DATA — ACCESS PLAN

Tool	Purpose	Access Plan
RSS feeds	Source new articles from trusted PM/product blogs	n8n RSS Feed Read node — no authentication required
LLM (OpenRouter)	Classify each article into a skill-area tag	Existing API key from FL-04
Local file	Store the deduplicated, tagged reading list (reading-list.md) and a seen-links log (seen-links.json)	n8n Read/Write File node — local filesystem, no authentication required

DRAFT INSTRUCTIONS

1. On trigger: fetch latest items from each configured RSS feed.
2. Compare fetched items against seen-links.json; discard anything already logged.
3. For each new item, classify it into one skill-area tag using the LLM (single word/phrase, not a summary).
4. Append new items to reading-list.md, grouped under their skill-area heading, with title, source, link, and date.
5. Update seen-links.json with the newly processed links.

6. If a feed is unreachable, skip it and continue processing the remaining feeds — one broken source must not stop the run.

EVAL CASES

1. New articles exist → correctly appended with title, source, link, date, and skill-area tag.
2. No new articles → nothing appended, file unchanged.
3. Same article appears again on a later run → correctly skipped, not duplicated.
4. One feed is broken or unreachable → remaining feeds still process; run completes.
5. Two feeds cover the same article → handled without duplicate or malformed entries.
6. An article's topic is ambiguous → agent still assigns its best single tag rather than failing the run.

RISKS & GUARDRAILS

> **Must never:** summarize, rewrite, or reproduce article content — only title, source, link, date, and a one-word/phrase skill tag are stored.
>
> **Must always:** keep the original source link visible and unaltered, so every entry can be independently verified.
>
> **Failure mode to design for:** a broken or slow feed must not block the rest of the run — skip and continue.

BUILD LOG

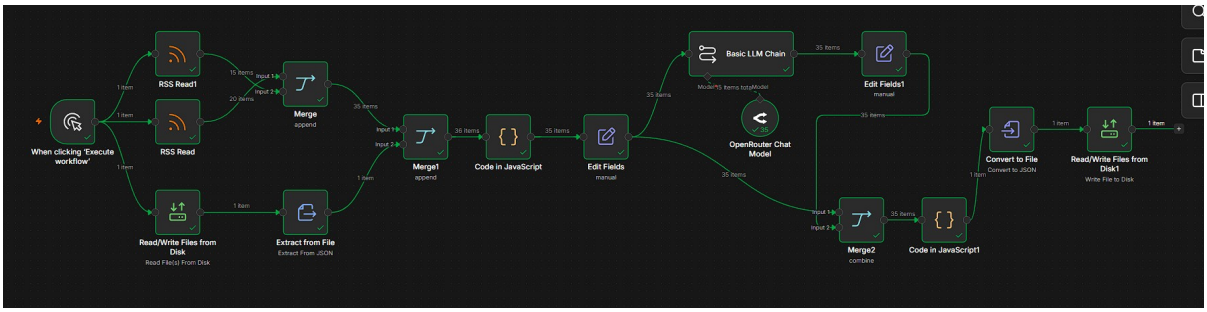
Real iteration as it happened — not a retroactive clean story.

Date	What I tried	What broke	What I changed / cut, and why
	Set up Google OAuth for Gmail + Sheets per FL-06 spec	Self-hosted n8n needed a manually configured Google Cloud OAuth client; high setup cost for a 10-hour build	Cut Gmail/Sheets entirely; pivoted to an RSS + local-file agent needing no OAuth (see Deviation section above)
	Built full node chain: dual RSS Read → Merge → read seen-links.json → dedup (Code node) → Edit Fields → LLM classification (OpenRouter) → Merge2 → write seen-links.json	Hit a “connection to the server was closed unexpectedly” error mid-build	Isolated the failing node using n8n's per-node ‘Execute step’ instead of running the full workflow; still narrowing down root cause
	Reviewed the seen-links write logic (Code in JavaScript1) against multi-run behavior	Found it rebuilds seenLinks only from the current run's articles — on a run with zero new articles, this overwrites the file with an empty list and erases all prior history	Fix identified (merge previous seenLinks with new ones instead of replacing) — not yet applied

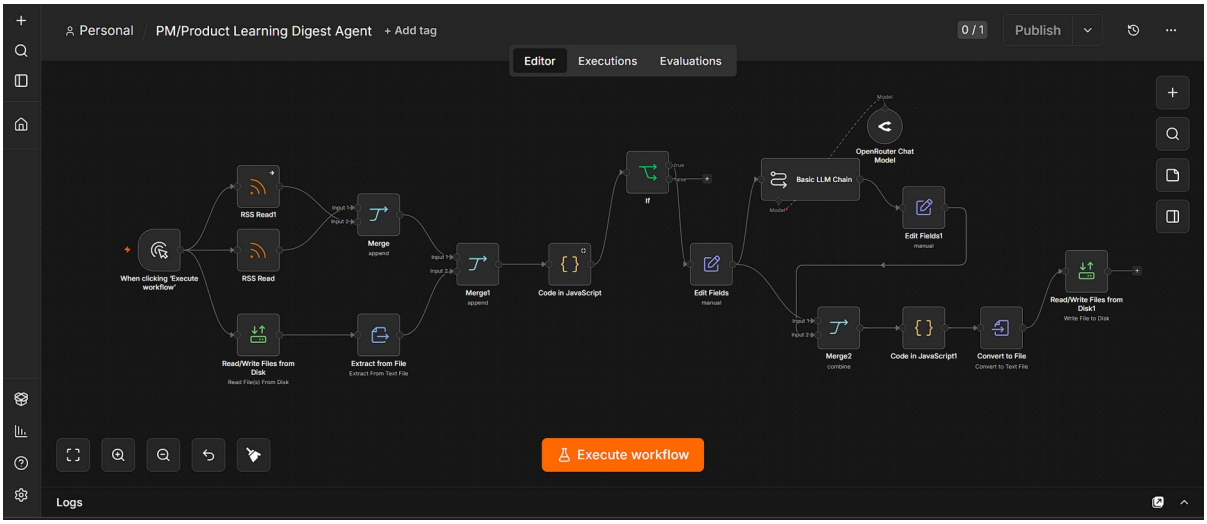
Date	What I tried	What broke	What I changed / cut, and why
	Ran the workflow end to end for the first time	All 15 + 20 = 35 articles across both feeds passed through dedup, classification, and the seen-links write with no node errors — first fully green run	Confirmed the pipeline itself works. Also confirmed a gap: no node writes a human-readable output file — classified articles currently have nowhere to land for me to actually read
	Rebuilt the dedup approach to read directly from reading-list.md instead of a separate seen-links.json, and added an If node to drop any RSS item with no title, plus onError handling on one RSS feed	Reading-list.md was still being overwritten each run instead of appended to — the same class of bug as before, just relocated to the new file	Fixed independently: Code in JavaScript1 now pulls the previous file content via the Extract from File node and prepends new entries above it with a divider, instead of replacing the file

EXECUTION EVIDENCE

First full end-to-end execution — all nodes completed successfully (green checkmarks), 35 total articles pulled across both RSS feeds, deduplicated, classified by skill area, and processed through to the tracking file.



Final architecture, after fixing the persistence bug: dedup now reads directly from reading-list.md (no separate seen-links file), an If node drops any RSS item with a missing title, and one RSS feed has onError handling so a broken source can't stop the run.



CURRENT STATUS & NEXT STEPS

> **Status:** both open issues resolved. Dedup now reads directly from reading-list.md, and the write step preserves prior entries instead of overwriting them — fixed independently after the first bug review.
>
> **Verification remaining:** run the workflow a second time to confirm persistence in practice — that new articles get appended and previously seen ones are correctly skipped and not lost.
>
> **Before final submission:** record the raw, unedited ~2 minute run capture showing the full loop — trigger → nodes complete → reading-list.md updated — once the second run is confirmed clean.

PLATFORM

n8n (self-hosted, local), continuing from FL-06. RSS Feed Read, HTTP Request (LLM classification), and Read/Write File nodes cover the full loop without any OAuth-based credentials.

SUBMISSION CHECKLIST

- Working agent — completes the full loop end to end without mid-run hand-editing. Both bugs fixed; recommend one more confirmation run before recording.
- Build log — filled in above with real entries as the build happened.
- Raw run capture — ~2 minute unedited screen recording of one successful end-to-end run, from trigger to result. Still to record.